CSA0926 – Programming in Java | CO3 Individual Assignment

1. HEADER

Name	K. SHALINI
Roll No.	17
Course Code & CO	CSA09, CO3 – Exception Handling and Multithreading
Assignment Set	Direct / Descriptive – Question 17
GitHub Repository	[https://github.com/Shalini387/CSA0926-JAVA/tree/main/CO3_AT2]
Submission Date	[11-08-2026]

Full Text of Assigned Question:

Q17. Validating Object Creation with Custom Exceptions — Write a Java program that models a Rectangle class whose constructor validates that both the length and width provided are positive numbers. If either value is zero or negative, the constructor should throw a custom exception named InvalidDimensionException. Demonstrate creating both valid and invalid Rectangle objects and handling the exception appropriately in each case.

Concepts: constructor validation, custom exceptions in OOP.

2. PROBLEM UNDERSTANDING

The program must validate object creation itself, not just method calls. A Rectangle should never exist in an invalid state, so the constructor checks that length and width are both positive before assigning them. If either value is zero or negative, a custom checked exception, InvalidDimensionException, is thrown instead of allowing the object to be created. This tests how custom exceptions can be used to enforce data integrity at the point of construction, and how the calling code must handle that exception to safely create objects.

3. APPROACH / DESIGN NOTES

- Created InvalidDimensionException by extending Exception (not RuntimeException), making it a checked exception so the caller is forced to handle it at compile time.
- Placed all validation logic inside the Rectangle constructor itself, so an invalid object can never be instantiated (fail-fast principle).
- Declared throws InvalidDimensionException on the constructor signature, since a constructor can propagate a checked exception the same way a method can.
- Kept the exception message dynamic, embedding the actual invalid length/width values so the caller gets useful diagnostic information rather than a generic error.
- Alternative considered: Using the built-in IllegalArgumentException, which is simpler since it's unchecked; rejected here since the assignment specifically requires a custom exception class.

4. SOURCE CODE

InvalidDimensionException.java

```
// Custom checked exception for invalid Rectangle dimensions
public class InvalidDimensionException extends Exception {

    public InvalidDimensionException(String message) {
        super(message);
    }
}
```

Rectangle.java

```
public class Rectangle {
    private double length;
    private double width;
```

```
    // Constructor validates length and width before creating the object
    public Rectangle(double length, double width) throws InvalidDimensionException {
        if (length <= 0 || width <= 0) {
            throw new InvalidDimensionException(
                "Invalid dimensions: length and width must be positive. " +
                "Given length = " + length + ", width = " + width);
        }
        this.length = length;
        this.width = width;
    }
    public double getArea() {
        return length * width;
    }

    public double getPerimeter() {
        return 2 * (length + width);
    }

    @Override
    public String toString() {
        return "Rectangle[length=" + length + ", width=" + width + "]";
    }
}
```

RectangleTest.java

```
public class RectangleTest {
    public static void main(String[] args) {
```

```
        // ---- Test Case 1: Valid rectangle ----
        System.out.println("=== Test Case 1: Valid Rectangle ===");
        try {
            Rectangle r1 = new Rectangle(10, 5);
            System.out.println("Created: " + r1);
            System.out.println("Area = " + r1.getArea());
            System.out.println("Perimeter = " + r1.getPerimeter());
        } catch (InvalidDimensionException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // ---- Test Case 2: Invalid - negative width ----
        System.out.println("\n=== Test Case 2: Negative Width ===");
        try {
            Rectangle r2 = new Rectangle(8, -3);
            System.out.println("Created: " + r2);
        } catch (InvalidDimensionException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // ---- Test Case 3: Invalid - zero length ----
        System.out.println("\n=== Test Case 3: Zero Length ===");
        try {
            Rectangle r3 = new Rectangle(0, 4);
            System.out.println("Created: " + r3);
        } catch (InvalidDimensionException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

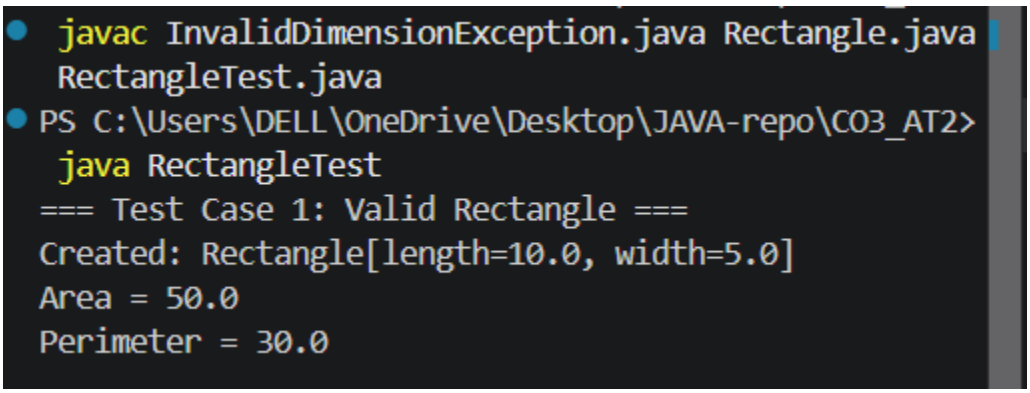
5. TEST CASES

#	Input (length, width)	Expected Output	Actual Output
1	(10, 5) — typical case	Rectangle created; Area = 50.0; Perimeter = 30.0	Created: Rectangle[length=10.0, width=5.0] Area = 50.0 Perimeter = 30.0
2	(8, -3) — edge case (negative width)	InvalidDimensionException thrown with given values in message	Error: Invalid dimensions: length and width must be positive. Given length = 8.0, width = -3.0
3	(0, 4) — boundary case (zero length)	InvalidDimensionException thrown with given values in message	Error: Invalid dimensions: length and width must be positive. Given length = 0.0, width = 4.0

Minimum three rows required: one normal/typical input, one edge case (negative value), and one boundary case (zero value).

6. SCREENSHOTS OF EXECUTION

Test Case 1 – Valid Rectangle



Test Case 2 – Negative Width

```
=== Test Case 2: Negative Width ===
Error: Invalid dimensions: length and width must be
positive. Given length = 8.0, width = -3.0
```

Test Case 3 – Zero Length

```
=== Test Case 3: Zero Length ===
Error: Invalid dimensions: length and width must be
positive. Given length = 0.0, width = 4.0
```

7. REFLECTION

While running this program, I understood why a constructor is a good place to enforce validation — it stops an invalid object from ever coming into existence, rather than catching bad data after the fact. One thing I had to be careful about was making InvalidDimensionException a checked exception by extending Exception rather than RuntimeException, since the assignment required the exception to be handled explicitly by the caller, not just optionally caught. I also made sure the exception message included the actual invalid values passed in, so the error output is genuinely useful rather than generic.

8. DECLARATION

I confirm that this submission — including the source code, design approach, and documentation — is my own work. AI assistance (Claude) was used to help structure the documentation and organize the test cases; the code logic was written, reviewed, and understood by me before submission.